6 On service discovery

This chapter covers

- Why service discovery is important to cloud-based applications
- The pros and cons of service discovery vs. the Load Balancer
- Setting up a Spring Netflix Eureka Server
- Registering a Spring Boot microservice with Eureka
- Using the Spring Cloud Load Balancer library for client-side load balancing

In any distributed architecture, we need to find the hostname or IP address of where a machine is located. This concept has been around since the beginning of distributed computing and is known formally as "service discovery." Service discovery can be something as simple as maintaining a property file with the addresses of all the remote services used by an application, or something as formalized as a Universal Description, Discovery, and Integration (UDDI) repository. Service discovery is critical to microservice, cloud-based applications for two key reasons:

- *Horizontal scaling or scale out*—This pattern usually requires adjustments in the application architecture, such as adding more instances of a service inside a cloud service and more containers.
- *Resiliency*—This pattern refers to the ability to absorb the impact of problems within an architecture or service without affecting the business. Microservice architectures need to be extremely sensitive to preventing a problem in a single service (or service instance) from cascading up and out to the consumers of the service.

First, service discovery allows the application team to quickly scale—horizontally—the number of service instances running in an environment. The service consumers are abstracted away from the physical location of the service. Because the service consumers don't know the physical location of the actual service instances, new service instances can be added or removed from the pool of available services.

This ability to quickly scale services without disrupting the service consumers is a compelling concept. It can move a development team that's used to building a monolithic, single-tenant (for example, one customer) application away from thinking about scaling only in terms of adding bigger and better hardware (vertical scaling) to the more robust approach to scaling by adding more servers with more services (horizontal scaling).

A monolithic approach usually drives development teams down the path of overbuying their capacity needs. Capacity increases come in clumps and spikes and are rarely a smooth, steady process. For example, consider the incremental number of requests made to e-commerce sites before some holidays. Microservices allow us to scale new service instances on demand. Service discovery helps abstract these deployments so that they occur away from the service consumer.

The second benefit of service discovery is that it helps increase application resiliency. When a microservice instance becomes unhealthy or unavailable, most service discovery engines remove that instance from their internal list of available services. The damage caused by a down service is minimized because the service discovery engine routes services around the unavailable service.

All of this may sound somewhat complicated, and you might be wondering why we can't use tried-and-true methods such as DNS (Domain Name Service) or a load balancer to help facilitate service discovery. Let's walk through why that won't work with a microservices-based application, particularly one that's running in the cloud. Then, we'll learn how to implement Eureka Discovery in our architecture.

6.1 Where's my service?

If you have an application that calls resources spread across multiple servers, it needs to find the physical location of those resources. In the non-cloud world, service location resolution was often solved through a combination of a DNS and a network load balancer (figure 6.1). In this traditional scenario, when an application needed to invoke a service located in another part of the organization, it attempted to invoke the service by using a generic DNS name along with a path that uniquely represented the service that the application wanted to invoke. The DNS name would resolve to a commercial load balancer such as the popular F5 load balancer (**http://f5.com**) or an open source load balancer such as HAProxy (**http://haproxy.org**).

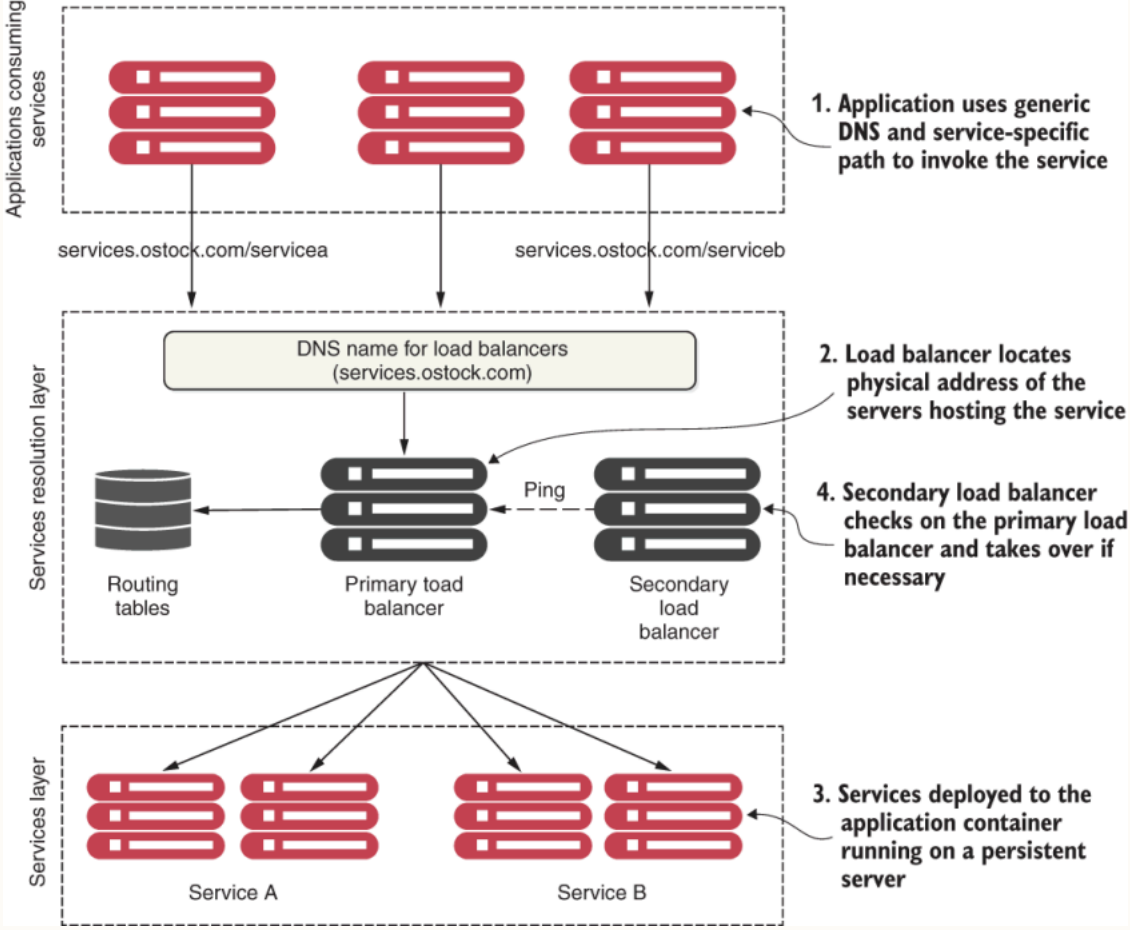


Figure 6.1 A traditional service location resolution model uses a DNS and a load balancer.

In the traditional scenario, the load balancer, upon receiving the request from the service consumer, located the physical address entry in a routing table based on the path the user was trying to access. This routing table entry contained a list of one or more servers hosting the service. The load balancer then picked one of the servers in the list and forwarded the request on to that server.

With this legacy model, each instance of a service was deployed in one or more application servers. The number of these application servers was often static (the number of application servers hosting a service didn't go up and down) and persistent (if a server running an application server crashed, it would be restored to the same state it was in at the time of the crash with the same IP address and configuration that it had previously). To achieve a form of high availability, a secondary idle load balancer pinged the primary load balancer to see if it was alive. If it wasn't alive, the secondary load balancer became active, taking over the IP address of the primary load balancer and began serving requests.

While this type of model works well with applications running inside the four walls of a corporate data center, and with a relatively small number of services running on a group of static servers, it doesn't work well for cloud-based microservice applications. The reasons for this include the following:

- *While the load balancer can be made highly available, it's a single point of failure for your entire infrastructure.* If the load balancer goes down, every application relying on it goes down too. While you can make a load balancer highly available, load balancers tend to be centralized chokepoints within your application infrastructure.
- *Centralizing your services into a single cluster of load balancers limits your ability to scale horizontally your load-balancing infrastructure across multiple servers.* Many commercial load balancers are constrained by two things: their redundancy model and their licensing costs.
  Most commercial load balancers use a hot-swap model for redundancy, so you only have a single server to handle the load, while the secondary load balancer is there only for failover in case the primary load balancer goes down. You are, in essence, constrained by your hardware. Commercial load balancers also have restrictive licensing models geared toward a fixed capacity rather than a more variable model.
- *Most traditional load balancers are statically managed.* They aren't designed for fast registration and deregistration of services. Traditional load balancers use a centralized database to store the routes for rules, and the only way to add new routes is often through the vendor's proprietary API.
- *Because a load balancer acts as a proxy to the services, service consumer requests need to have them mapped to the physical services.* This translation layer often adds another layer of complexity to your service infrastructure because the mapping rules for the service have to be defined and deployed by hand. Also, in a traditional load balancer scenario, the registration of new service instances is not done when a new service instance starts.

These four reasons aren't a general indictment of load balancers. A load balancer works well in a corporate environment where the size and scale of most applications can be handled through a centralized network infrastructure. But load balancers still have a role to play in centralizing SSL termination and managing service port security. A load balancer can lock down inbound (ingress) and outbound (egress) port access to all the servers sitting behind it. This concept of "least network access" is often a critical component when trying to meet industry-standard certification requirements such as PCI (Payment Card Industry) compliance.

However, in the cloud, where you have to deal with massive amounts of transactions and redundancy, a centralized piece of network infrastructure doesn't ultimately work as well. This is because it doesn't scale effectively and isn't cost effective. Let's now look at how you can implement a robust service discovery mechanism for your cloud-based applications.

6.2 Service discovery in the cloud

The solution for a cloud-based microservice environment is to use a service discovery mechanism that's

- *Highly available*—Service discovery needs to be able to support a "hot" clustering environment where service lookups can be shared across multiple nodes in a service discovery cluster. If a node becomes unavailable, other nodes in the cluster should be able to take over.
  A cluster can be defined as a group of multiple server instances. All instances of this environment have an identical configuration and work together to provide high availability, reliability, and scalability. A cluster combined with a load balancer can offer failover to prevent service interruptions and session replication to store session data.
- *Peer-to-peer*—Each node in the service discovery cluster shares the state of a service instance.
- *Load balanced*—Service discovery needs to dynamically load balance requests across all service instances. This ensures that the service invocations are spread across all the service instances managed by it. In many ways, service discovery replaces the more static, manually managed load balancers found in many early web application implementations.
- *Resilient*—The service discovery's client should cache service information locally. Local caching allows for gradual degradation of the service discovery feature so that if the service discovery service becomes unavailable, applications can still function and locate the services based on the information maintained in their local cache.
- *Fault tolerant*—Service discovery needs to detect when a service instance isn't healthy and remove that instance from the list of available services that can take client requests. It should detect these faults with services and take action without human intervention.

In the following sections, we're going to

- Walk you through the conceptual architecture of how a cloud-based service discovery agent works
- Show you how client-side caching and load balancing allows the service to continue to function even when the service discovery agent is unavailable
- Show you how to implement service discovery using Spring Cloud and Netflix's Eureka service discovery agents

6.2.1 The architecture of service discovery

To begin our discussion around service discovery, we need to understand four concepts. These general concepts are often shared across all service discovery implementations:

- *Service registration*—How a service registers with the service discovery agent
- *Client lookup of service address*—How a service client looks up service information
- *Information sharing*—How nodes share service information
- *Health monitoring*—How services communicate their health back to the service discovery agent

The principal objective of service discovery is to have an architecture where our services indicate where they are physically located instead of having to manually configure their location. Figure 6.2 shows how service instances are added and removed, and how they update the service discovery agent and become available to process user requests.

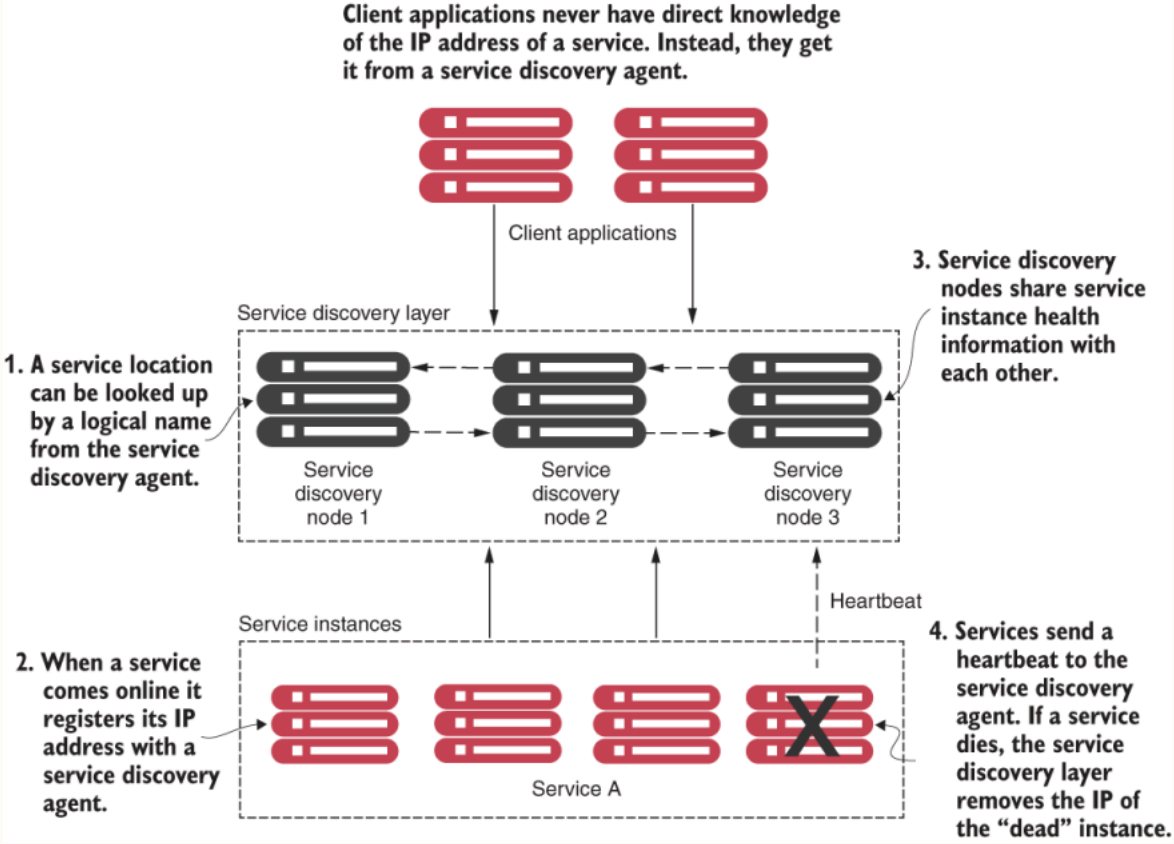


Figure 6.2 As service instances are added or removed, the service discovery nodes are updated and made available to process user requests.

Figure 6.2 shows the flow of the previous four bulleted points (service registration, service discovery lookup, information sharing, and health monitoring) and what typically occurs when we implement a service discovery pattern. In the figure, one or more service discovery nodes have started. These service discovery instances usually don't have a load balancer that sits in front of them.

As service instances start, they'll register their physical location, path, and port that one or more service discovery instances can use to access the instances. While each instance of a service has a unique IP address and port, each service instance that comes up registers under the same service ID. A service ID is nothing more than a key that uniquely identifies a group of the same service instances.

A service usually only registers with one service discovery service instance. Most service discovery implementations use a peer-to-peer model of data propagation, where the data around each service instance is communicated to all the other nodes in the cluster. Depending on the service discovery implementation, the propagation mechanism might use a hardcoded list of services to propagate to or use a multicasting protocol like the gossip or infection-style protocol to allow other nodes to "discover" changes in the cluster.

NOTE If you are interested in knowing more about the gossip or infection-style protocols, we highly recommend you review the following: Consul's "Gossip Protocol" (**https://www.consul.io/docs/internals/gossip.html**) or Brian Storti's post, "SWIM: The scalable membership protocol" (**https://www.brianstorti.com/swim/)**.

Finally, each service instance pushes to or pulls from its status by the service discovery service. Any services failing to return a good health check are removed from the pool of available service instances. Once a service is registered with a service discovery service, it's ready to be used by an application or service that needs to make use of its capabilities. Different models exist for a client to discover a service.

As a first approach, the client relies solely on the service discovery engine to resolve service locations each time a service is called. With this approach, the service discovery engine is invoked each time a call to a registered microservice instance is made. Unfortunately, this approach is brittle because the service client is completely dependent on the service discovery engine to find and invoke a service.

A more robust approach uses what's called *client-side load balancing*. This mechanism uses an algorithm like zone-specific or round-robin to invoke the instances of the calling services. When we say "round-robin algorithm load balancing," we are referring to a way of distributing client requests across several servers. This consists of forwarding a client request to each of the servers in turn. An advantage of using the client-side load balancer with Eureka is that when a service instance goes down, it is removed from the registry. Once that is done, the client-side load balancer updates itself without manual intervention by establishing constant communication with the registry service. Figure 6.3 illustrates this approach.

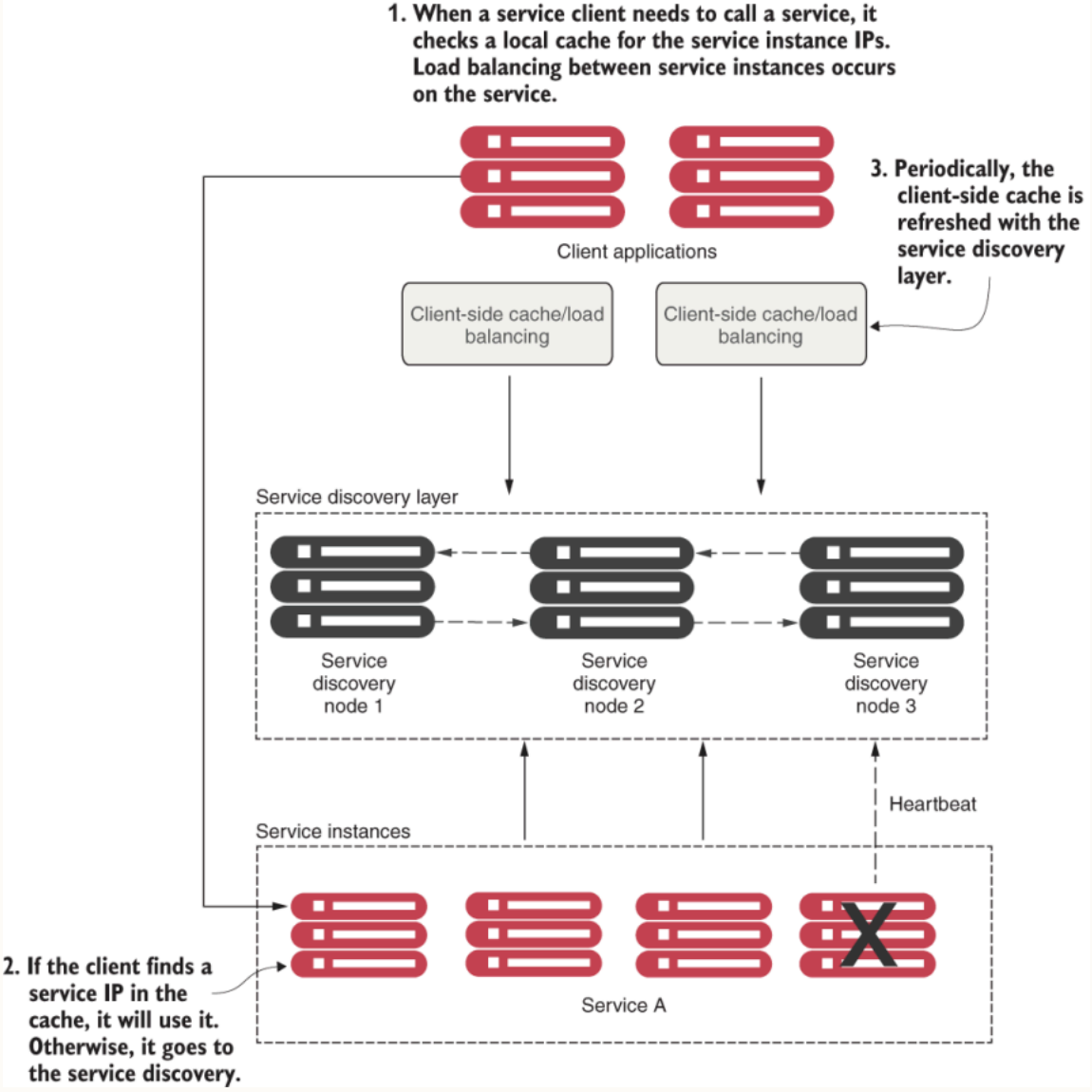


Figure 6.3 Client-side load balancing caches the location of the services so that the service client doesn't need to contact service discovery on every call.

In this model, when a consuming client needs to invoke a service

1. It contacts the discovery service for all the instances a service consumer (client) is asking for and then caches data locally on the service consumer's machine.
2. Each time a client wants to call the service, the service consumer looks up the location information for the service from the cache. Usually, client-side caching will use a simple load-balancing algorithm like the round-robin load-balancing algorithm to ensure that service calls are spread across multiple service instances.
3. The client then periodically contacts the discovery service and refreshes its cache of service instances. The client cache is eventually consistent, but there's always a risk that when the client contacts the service discovery instance for a refresh and calls are made, calls might be directed to a service instance that isn't healthy.

If during the course of calling a service, the service call fails, the local service discovery cache is invalidated and the service discovery client will attempt to refresh its entries from the service discovery agent. Let's now take the generic service discovery pattern and apply it to our O-stock problem domain.

6.2.2 Service discovery in action using Spring and Netflix Eureka

In this section, we will implement service discovery by setting up a service discovery agent and then register two services with the agent. With this implementation, we'll use the information retrieved by the service discovery to call a service from another service. Spring Cloud offers multiple methods for looking up information from a service discovery agent. We'll walk through the strengths and weaknesses of each approach.

Again, Spring Cloud makes this type of setup trivial to undertake. We'll use Spring Cloud and Netflix's Eureka Service Discovery engine to implement your service discovery pattern. For the client-side load balancing, we'll use the Spring Cloud Load Balancer.

NOTE In this chapter, we are not going to use Ribbon. Ribbon was the de facto client-side load balancer for REST-based communications among applications using Spring Cloud. Although Netflix Ribbon client-side load balancing was a stable solution, it has now entered a maintenance mode, so unfortunately, it will not be developed anymore.

In this section, we will explain how to use the Spring Cloud Load Balancer, which is a replacement for Ribbon. Currently, the Spring Cloud Load Balancer is still under active development, so expect new functionalities soon. In the previous two chapters, we kept our licensing service simple and included the organization's name for the licenses with the license data. In this chapter, we'll break the organization information into its own service. Figure 6.4 shows the implementation of the client-side caching with Eureka for our O-stock microservices.

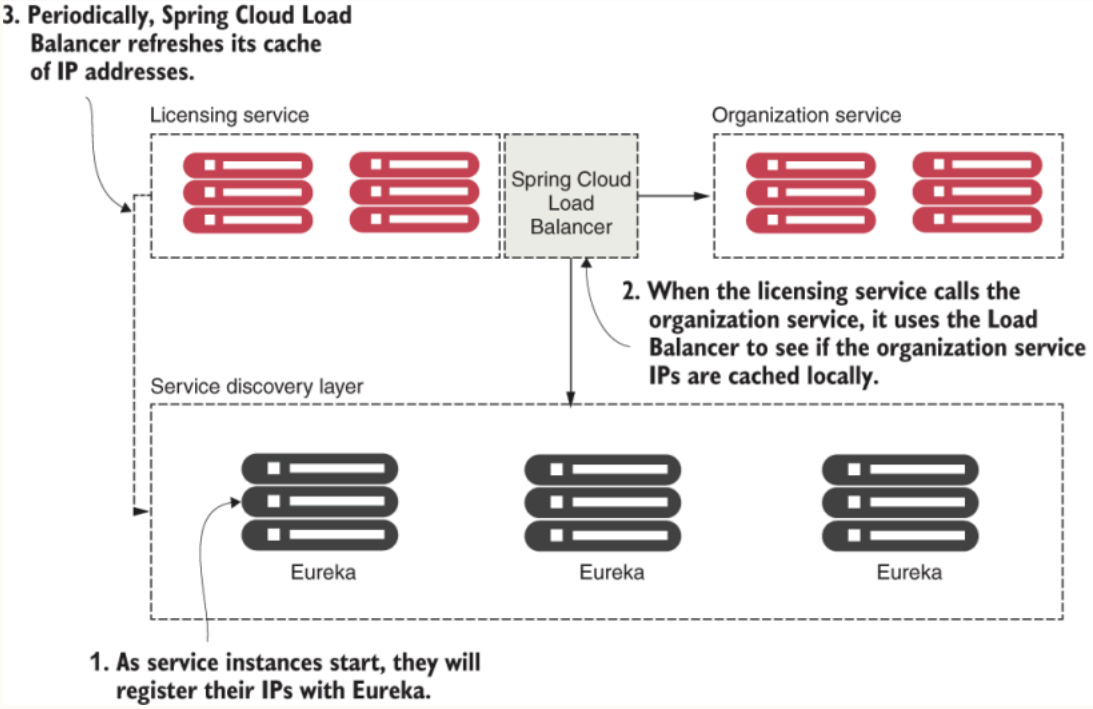


Figure 6.4 By implementing client-side caching and Eureka with O-stock's licensing and organization services, you can lessen the load on the Eureka Servers and improve client stability if Eureka becomes unavailable.

When the licensing service is invoked, it will call the organization service to retrieve the organization information associated with the designated organization ID. The actual resolution of the organization service's location is held in a service discovery registry. For this example, we'll register two instances of the organization service with a service discovery registry and then use client-side load balancing to look up and cache the registry in each service instance. Figure 6.4 shows this arrangement:

1. As the services are bootstrapped, the licensing and organization services register with the Eureka service. This registration process tells Eureka the physical location and port number of each service instance, along with a service ID for the service being started.
2. When the licensing service calls to the organization service, it uses the Spring Cloud Load Balancer to provide client-side load balancing. The Load Balancer contacts the Eureka service to retrieve service location information and then caches it locally.
3. Periodically, the Spring Cloud Load Balancer will ping the Eureka service and refresh its local cache of service locations.

Any new organization service instance is now visible to the licensing service locally, while any unhealthy instances are removed from the local cache. We'll implement this design by setting up our Spring Cloud Eureka service.

6.3 Building our Spring Eureka service

In this section, we'll set up our Eureka service using Spring Boot. Like the Spring Cloud Config service, setting up a Spring Cloud Eureka service starts with building a new Spring Boot project and applying annotations and configurations. Let's begin by creating this project with the Spring Initializr (**https://start.spring.io/**). To achieve this, we'll follow these steps in the Spring Initializr:

1. Select Maven as the project type.
2. Select Java as the language.
3. Select the 2.2.x latest or a more stable Spring version.
4. Write `com.optimagrowth` as the group and `eurekaserver` as the artifact.
5. Expand the options list and write `Eureka Server` as the name, `Eureka server` as the description, and `com.optimagrowth.eureka` as its package name.
6. Select JAR Packaging.
7. Select Java 11 as the Java version.

8. Add the Eureka Server, Config Client, and Spring Boot Actuator dependencies as shown in figure 6.5. Listing 6.1 shows the Eureka Server pom.xml file.

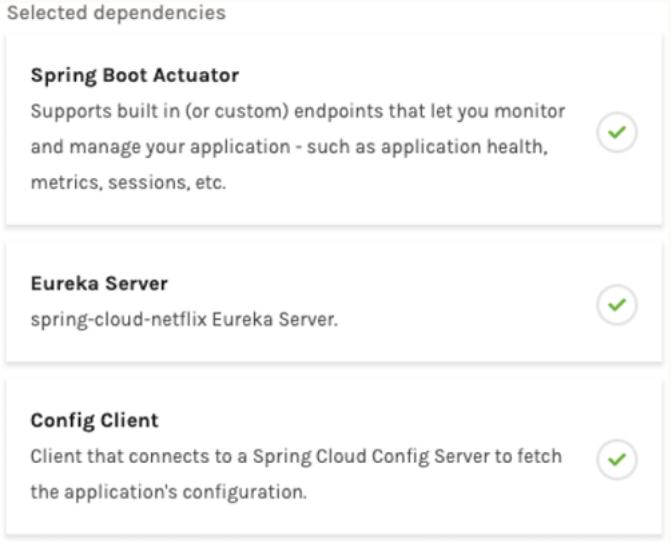


Figure 6.5 Eureka Server dependencies in Spring Initializr

Listing 6.1 Maven pom file for the Eureka Server

```
<?xml version="1.0" encoding="UTF-8"?>
<project xmlns="http://maven.apache.org/POM/4.0.0"
 xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
 xsi:schemaLocation="http://maven.apache.org/POM/4.0.0
 https://maven.apache.org/xsd/maven-4.0.0.xsd">
   <modelVersion>4.0.0</modelVersion>
   <parent>
     <groupId>org.springframework.boot</groupId>
     <artifactId>spring-boot-starter-parent</artifactId>
     <version>2.2.5.RELEASE</version>
     <relativePath/> <!-- lookup parent from repository -->
   </parent>
   <groupId>com.optimagrowth</groupId>
   <artifactId>eurekaserver</artifactId>
   <version>0.0.1-SNAPSHOT</version>
   <name>Eureka Server</name>
   <description>Eureka Server</description>

   <properties>
     <java.version>11</java.version>
     <spring-cloud.version>Hoxton.SR1</spring-cloud.version>
   </properties>

   <dependencies>
     <dependency>
       <groupId>org.springframework.boot</groupId>
       <artifactId>spring-boot-starter-actuator</artifactId>
     </dependency>
     <dependency>
       <groupId>org.springframework.cloud
       </groupId>                                    (1)
       <artifactId>spring-cloud-starter-config</artifactId>
     </dependency>
     <dependency>
       <groupId>org.springframework.cloud</groupId>
       <artifactId>spring-cloud-starter-netflix-eureka-server
       </artifactId>                                 (2)
       <exclusions>                                  (3)
         <exclusion>
           <groupId>org.springframework.cloud</groupId>
           <artifactId>spring-cloud-starter-ribbon</artifactId>
         </exclusion>
         <exclusion>
```

```
            <groupId>com.netflix.ribbon</groupId>
            <artifactId>ribbon-eureka</artifactId>
          </exclusion>
        </exclusions>
      </dependency>
      <dependency>
        <groupId>org.springframework.cloud</groupId>
        <artifactId>spring-cloud-starter-loadbalancer
        </artifactId>                                    ❹
      </dependency>
      <dependency>
        <groupId>org.springframework.boot</groupId>
        <artifactId>spring-boot-starter-test</artifactId>
        <scope>test</scope>
        <exclusions>
          <exclusion>
            <groupId>org.junit.vintage</groupId>
            <artifactId>junit-vintage-engine</artifactId>
          </exclusion>
        </exclusions>
      </dependency>
    </dependencies>

<!--Rest of pom.xml omitted for conciseness
...
</project>
```

❶ Tells Maven to include the client that connects to a Spring ConfigServer to retrieve the application's configuration

❷ Tells Maven to include the Eureka libraries

❸ Excludes the Netflix Ribbon libraries

❹ Tells Maven to include the Spring Cloud Load Balancer libraries

The next step is to set up the src/main/resources/bootstrap.yml file with the settings we need to retrieve the configuration from the Spring Config Server previously created in chapter 5. We also need to add the configuration to disable Ribbon as our default client-side load balancer. The following listing shows how your bootstrap.yml file should look.

Listing 6.2 Setting up the Eureka bootstrap.yml file

```
spring:
  application:
    name: eureka-server            ❶
  cloud:
    config:
      uri: http://localhost:8071   ❷

    loadbalancer:                  ❸
      ribbon:
        enabled: false
```

❶ Names the Eureka service so the Spring Cloud Config client knows which service it's looking up

❷ Specifies the location of the Spring Cloud Config Server

❸ Because Ribbon is still the default client-side load balancer, we disable it using the loadbalancer.ribbon.enabled configuration.

Once we add the Spring Configuration Server information in the bootstrap file on the Eureka Server and we disable Ribbon as our load balancer, we can continue with the next step. That step adds the configuration needed to set up the Eureka service running in stand-alone mode (no other nodes in the cluster) in the Spring Configuration Server.

In order to achieve this, we must create the Eureka Server configuration file in the repository we set up in the Spring Config service. (Remember, we can specify a repository as a classpath, filesystem, GIT, or Vault.) The configuration file should be named as the `spring.application.name` property previously defined in the Eureka bootstrap.yml file of the Eureka service. For purposes of this exam-

ple, we will create the eureka-server.yml file in classpath/configserver/src/main/resources/config/eureka-server.yml. Listing 6.3 shows the contents of this file.

NOTE If you didn't follow the code listings in chapter 5, you can download the code from this link: https://github.com/ihuaylupo/manning-smia/tree/master/chapter5.

Listing 6.3 Setting up the Eureka configuration in the Spring Config Server

```
server:
  port: 8070                                          ❶
eureka:
  instance:
    hostname: localhost                               ❷
  client:
    registerWithEureka: false                         ❸
    fetchRegistry: false                              ❹
    serviceUrl:
      defaultZone:                                    ❺
        http://${eureka.instance.hostname}:${server.port}/eureka/
  server:
    waitTimeInMsWhenSyncEmpty: 5                      ❻
```

❶ Sets the listening port for the Eureka Server

❷ Sets the Eureka instance hostname

❸ Tells the Config Server to not register with the Eureka service ...

❹ ... and to not cache registry information locally

❺ Provides the service URL

❻ Sets the initial time to wait before the server takes requests

The key properties set in listing 6.3 are as follows:

- `server.port` —Sets the default port.
- `eureka.instance.hostname` —Sets the Eureka instance hostname for the Eureka service.
- `eureka.client.registerWithEureka` —Tells the Config Server not to register with Eureka when the Spring Boot Eureka application starts.
- `eureka.client.fetchRegistry` —When set to false, tells the Eureka service that as it starts, it doesn't need to cache its registry information locally. When running a Eureka client, you'll want to change this value for the Spring Boot services that are going to register with Eureka.
- `eureka.client.serviceUrl.defaultZone` —Provides the service URL for any client. It is a combination of the `eureka.instance.hostname` and the `server.port` attributes.
- `eureka.server.waitTimeInMsWhenSyncEmpty` —Sets the time to wait before the server takes requests.

You'll notice that the last attribute in listing 6.3, `eureka.server.waitTimeInMsWhen-SyncEmpty`, indicates the time to wait in milliseconds before starting. When you're testing your service locally, you should use this line because Eureka won't immediately advertise any services that register with it. By default, it waits 5 minutes to give all of the services a chance to register with it before advertising them. Using this line for local testing helps to speed up the amount of time it takes for the Eureka service to start and to show the services registered with it.

NOTE Individual services registering with Eureka take up to 30 seconds to show up in the Eureka service. That's because Eureka requires three consecutive heartbeat pings from the service, which are spaced 10 seconds apart, before it will say the service is ready for use. Keep this in mind as you're deploying and testing your own services.

The last piece of setup work you need to do for your Eureka service is to add an annotation to the application bootstrap class you use to start your Eureka service. For the Eureka service, you can find the application bootstrap class, `EurekaServerApplication`, in the src/main/java/com/optimagrowth/eureka/EurekaServerApplication.java class file. The following listing shows where to add the annotations.

Listing 6.4 Annotating the bootstrap class to enable the Eureka Server

```
package com.optimagrowth.eureka;

import org.springframework.boot.SpringApplication;
import org.springframework.boot.autoconfigure.SpringBootApplication;
import org.springframework.cloud.netflix.eureka.server.EnableEurekaServer;

@SpringBootApplication
@EnableEurekaServer                    (1)
public class EurekaServerApplication {

   public static void main(String[] args) {
     SpringApplication.run(EurekaServerApplication.class, args);
   }

}
```

(1) Enables the Eureka Server in the Spring service

At this point, we only use a new annotation, `@EnableEurekaServer`, to enable our service as an Eureka service. Now we can start the Eureka service by running the `mvn spring-boot:run` or run `docker-compose` commands. Once the startup command is executed, we should have a running Eureka service with no services registered in it. We first need to run the Spring Config service because it contains the Eureka application configuration. If you don't run your configuration service first, you will get the following error:

```
Connect Timeout Exception on Url - http://localhost:8071.
Will be trying the next url if available.
    com.sun.jersey.api.client.ClientHandlerException: java.net.ConnectException:
    Connection refused (Connection refused)
```

To avoid the previous issue, try running the services with Docker Compose. Remember, you can find the docker-compose.yml file updated in the chapter repository on GitHub. Now, let's move on to building out the organization service. Then we will register the licensing and the organization services with our Eureka service.

6.4 Registering services with Spring Eureka

At this point, we have a Spring-based Eureka Server up and running. In this section, we'll configure the organization and licensing services to register themselves with our Eureka Server. This work is done in preparation for having a service client look up a service from our Eureka registry. By the time we're done with this section, you should have a firm understanding of how to register a Spring Boot microservice with Eureka.

Registering a Spring Boot-based microservice with Eureka is a straightforward exercise. For the purposes of this chapter, we're not going to walk through all of the Java code involved with writing the service (we purposely kept that amount of code small), but instead, focus on registering the service with the Eureka service registry you created in the previous section.

In this section, we introduce a new service that we'll call the organization service. This service will contain the CRUD endpoints. You can download the code for the licensing and organization services from this link:

https://github.com/ihuaylupo/manning-smia/tree/master/chapter6/Initial

NOTE At this point, you can use other microservices you might have. Just pay attention to the service ID names as you register them with service discovery.

The first thing we need to do is to add the Spring Eureka dependency to our organization and licensing services' pom.xml files. The following listing shows how.

Listing 6.5 Adding the Spring Eureka dependency to the organization's service pom.xml

```
<dependency>
  <groupId>org.springframework.cloud</groupId>
  <artifactId>                         (1)
    spring-cloud-starter-netflix-eureka-client
  </artifactId>
</dependency>
```

❶ Includes the Eureka libraries so that the service can register with Eureka

The `spring-cloud-starter-netflix-eureka-client` artifact holds the JAR files that Spring Cloud uses to interact with your Eureka service. After we've set up the pom.xml file, we need to make sure we have set the `spring.application.name` in the bootstrap.yml file of the service we want to register. The following listings, 6.6 and 6.7, indicate how to do this.

Listing 6.6 Adding the spring.application.name to the organization service

```
spring:
  application:
    name: organization-service      ❶
    profiles:
      active: dev
  cloud:
    config:
      uri: http://localhost:8071
```

❶ Logical name of the service that will be registered with Eureka

Listing 6.7 Adding the spring.application.name to the licensing service

```
spring:
  application:
    name: licensing-service      ❶
    profiles:
      active: dev
  cloud:
    config:
      uri: http://localhost:8071
```

❶ Logical name of the service that will be registered with Eureka

Every service registered with Eureka will have two components associated with it: the application ID and the instance ID. The application ID represents a group service instance. In a Spring Boot microservice, the application ID is always the value set by the `spring.application.name` property. For our organization service, this property is creatively named `organization-service`, and for our licensing service, it's named `licensing-service`. The instance ID will be a randomly autogenerated number to represent a single service instance.

Next, we need to tell Spring Boot to register the organization and licensing services with Eureka. This registration is done via additional configuration in the service's configuration files managed by the Spring Config service. For this example, these files are located in the following two files for the Spring Configuration Server project. Listing 6.8 then shows how to register the services with Eureka.

- src/main/resources/config/organization-service.properties
- src/main/resources/config/licensing-service.properties

NOTE Remember, the configuration file can be either a YAML or a properties file and can be located in the classpath, filesystem, Git repository, or Vault. It depends on the configuration you've set in the Spring Config Server. For this example, we selected the classpath and properties file, but feel free to make the changes that best suit your needs.

Listing 6.8 Modifying the service application.properties files for Eureka

```
eureka.instance.preferIpAddress = true     ❶

eureka.client.registerWithEureka = true    ❷

eureka.client.fetchRegistry = true         ❸

eureka.client.serviceUrl.defaultZone =
    http://localhost:8070/eureka/          ❹
```

❶ Registers the IP address of the service rather than the server name

❷ Registers the service with Eureka

❸ Pulls down a local copy of the registry

❹ Sets the location of the Eureka service

If you have an application.yml file, your file should look like that shown in the following code to register the services with Eureka. The `eureka.instance.preferIpAddress` property tells Eureka that you want to register the service's IP address with Eureka rather than its hostname.

```
eureka:
  instance:
    preferIpAddress: true
  client:
    registerWithEureka: true
    fetchRegistry: true
    serviceUrl: defaultZone: http://localhost:8070/eureka/
```

Why prefer an IP address?

By default, Eureka registers the services that contact it by hostname. This works well in a server-based environment, where a service is assigned a DNS-backed hostname. However, in a container-based deployment (for example, Docker), containers are started with randomly generated hostnames and no DNS entries for the containers. If you don't set the `eureka.instance.preferIpAddress` to true, your client applications won't correctly resolve the location of the hostnames; there will be no DNS entry for that container. Setting the `preferIpAddress` attribute informs the Eureka service that the client wants to be advertised by IP address.

Personally, we always set this attribute to `true`. Cloud-based microservices are supposed to be ephemeral and stateless. These can be started up and shut down at will, so IP addresses are more appropriate for these types of services.

The `eureka.client.registerWithEureka` attribute is the trigger to tell the organization and the licensing services to register with Eureka. The `eureka.client .fetchRegistry` attribute tells the Spring Eureka client to fetch a local copy of the registry. Setting this attribute to `true` caches the registry locally instead of calling the Eureka service with each lookup. Every 30 seconds, the client software recontacts the Eureka service for any changes to the registry.

NOTE These two properties are set by default to `true`, but we've included the properties in the application configuration's file for illustrative purposes only. The code will work without setting those properties to `true`.

The last attribute, `eureka.serviceUrl.defaultZone`, holds a comma-separated list of Eureka services the client uses to resolve to service locations. For our purposes, we're only going to have one Eureka service. We can also declare all the key-value properties defined previously in the bootstrap file of each service. But the idea is to delegate the configuration to the Spring Config service. That's why we register all the configurations in the service configuration file in our Spring Config service repository. So far, the bootstrap file of these services should only contain the application name, a profile (if needed), and the Spring Cloud configuration URI.

Eureka and high availability

Setting up multiple URL services isn't enough for high availability. The `eureka .service-Url.defaultZone` attribute only provides a list of Eureka services for the client to communicate with. You also need to set up the Eureka services to replicate the contents of their registries with each other. A group of Eureka registries communicate with each other using a peer-to-peer communication model, where each Eureka service must be configured so that it knows about the other nodes in the cluster.

Setting up a Eureka cluster is outside of the scope of this book. If you're interested in setting up a Eureka cluster, visit the Spring Cloud project's website for further information:

https://projects.spring.io/spring-cloud/spring-cloud.html#spring-cloud-eureka-server

At this point, we have two services registered with our Eureka service. We can use Eureka's REST API or the Eureka dashboard to see the contents of the registry. We'll explain each in the following sections.

6.4.1 Eureka's REST API

To see all the instances of a service in the REST API, select the following GET endpoint:

```
http://<eureka service>:8070/eureka/apps/<APPID>
```

For instance, to see the organization service in the registry, you would call the following endpoint: `http://localhost:8070/eureka/apps/organization-service`. Figure 6.6 shows the response.

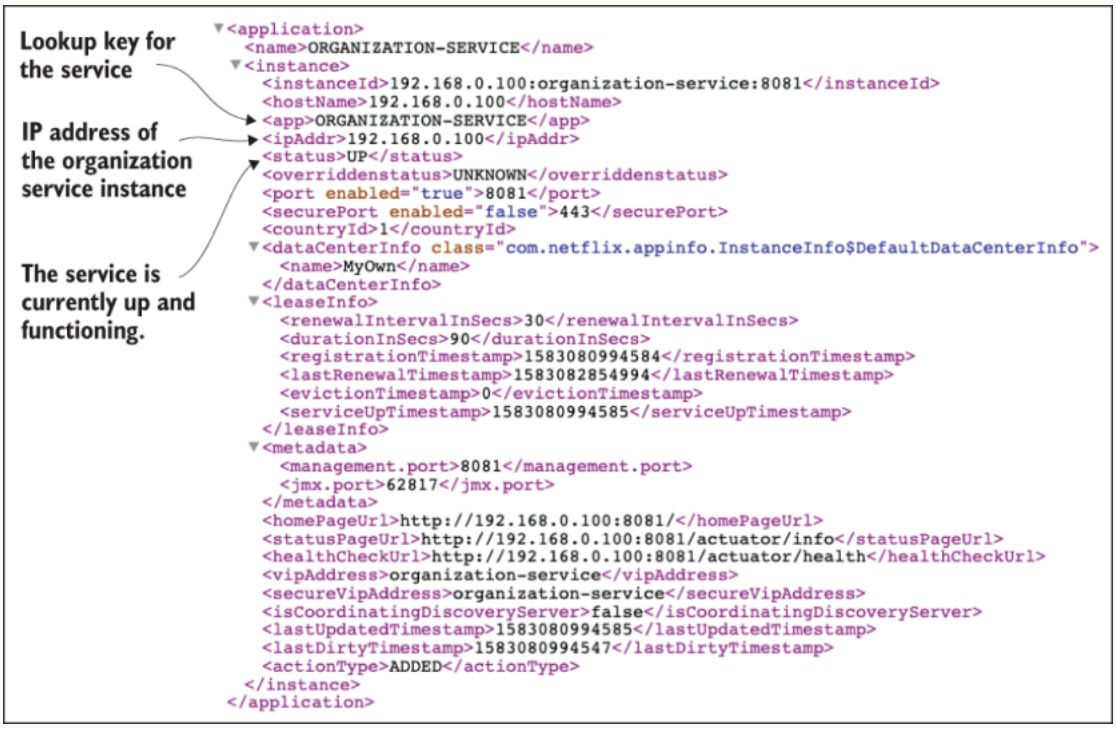

Lookup key for the service
IP address of the organization service instance
The service is currently up and functioning.
<application>
<name>ORGANIZATION-SERVICE</name>
<instance>
<instanceId>192.168.0.100:organization-service:8081</instanceId>
<hostName>192.168.0.100</hostName>
<app>ORGANIZATION-SERVICE</app>
<ipAddr>192.168.0.100</ipAddr>
<status>UP</status>
<overriddenstatus>UNKNOWN</overriddenstatus>
<port enabled="true">8081</port>
<securePort enabled="false">443</securePort>
<countryId>1</countryId>
<dataCenterInfo class="com.netflix.appinfo.InstanceInfo$DefaultDataCenterInfo">
<name>MyOwn</name>
</dataCenterInfo>
<leaseInfo>
<renewalIntervalInSecs>30</renewalIntervalInSecs>
<durationInSecs>90</durationInSecs>
<registrationTimestamp>1583080994584</registrationTimestamp>
<lastRenewalTimestamp>1583082854994</lastRenewalTimestamp>
<evictionTimestamp>0</evictionTimestamp>
<serviceUpTimestamp>1583080994585</serviceUpTimestamp>
</leaseInfo>
<metadata>
<management.port>8081</management.port>
<jmx.port>62817</jmx.port>
</metadata>
<homePageUrl>http://192.168.0.100:8081/</homePageUrl>
<statusPageUrl>http://192.168.0.100:8081/actuator/info</statusPageUrl>
<healthCheckUrl>http://192.168.0.100:8081/actuator/health</healthCheckUrl>
<vipAddress>organization-service</vipAddress>
<secureVipAddress>organization-service</secureVipAddress>
<isCoordinatingDiscoveryServer>false</isCoordinatingDiscoveryServer>
<lastUpdatedTimestamp>1583080994585</lastUpdatedTimestamp>
<lastDirtyTimestamp>1583080994547</lastDirtyTimestamp>
<actionType>ADDED</actionType>
</instance>
</application>
```

Figure 6.6 The Eureka REST API showing the organization service. The response shows the IP address of the service instances registered in Eureka, along with the service status.

The default format returned by the Eureka service is XML. Eureka can also return the data in figure 6.6 as a JSON payload, but you'll have to set the Accept HTTP header to `application/json`. An example of the JSON payload is shown in figure 6.7.
```

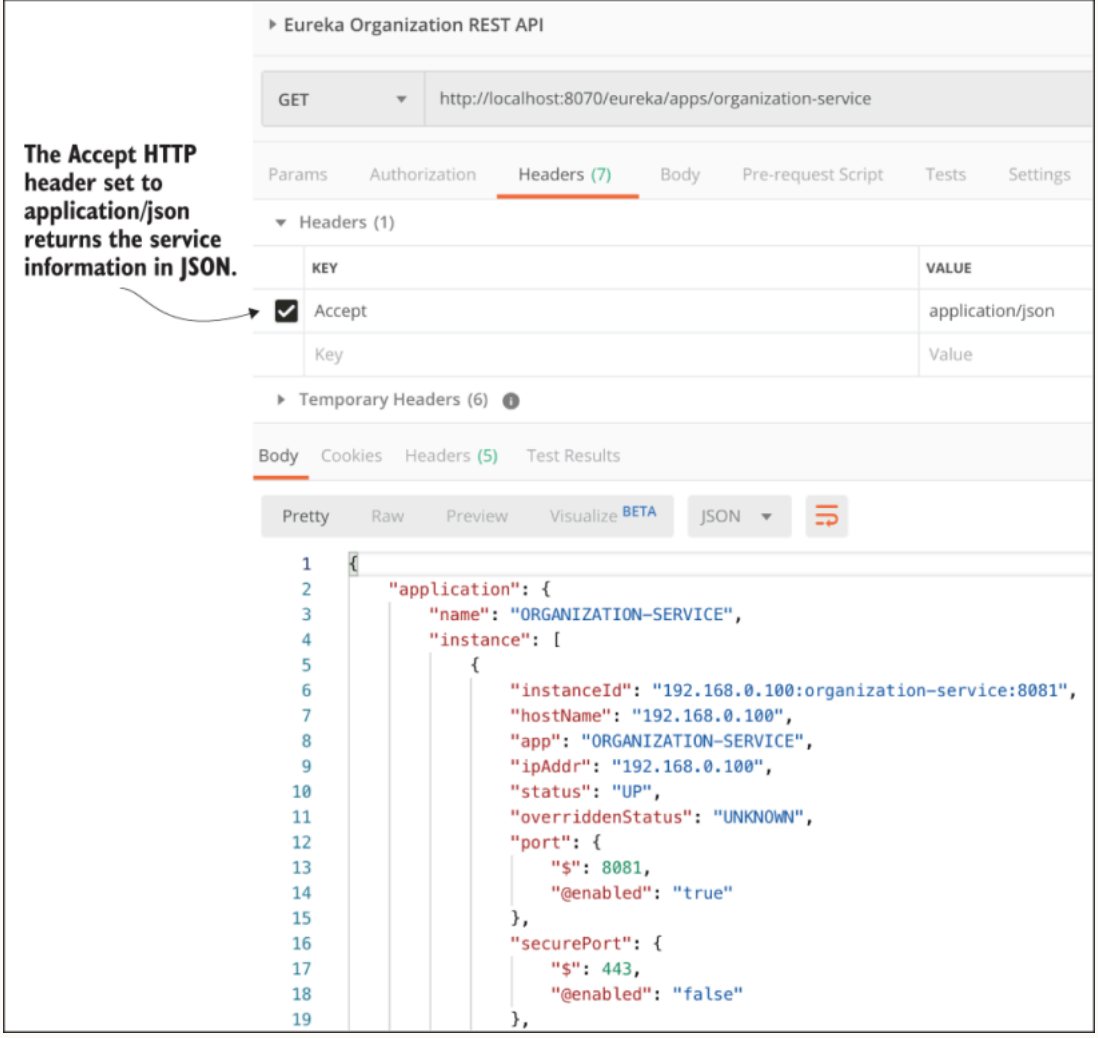


Figure 6.7 The Eureka REST API with JSON results

6.4.2 Eureka dashboard

Once the Eureka service is up, we can point our browsers to http://localhost:8070 to view the Eureka dashboard. The Eureka dashboard allows us to see the registration status of our services. Figure 6.8 shows an example of the Eureka dashboard.

Now, that we've registered the organization and licensing services, let's see how we can use service discovery to look up a service.

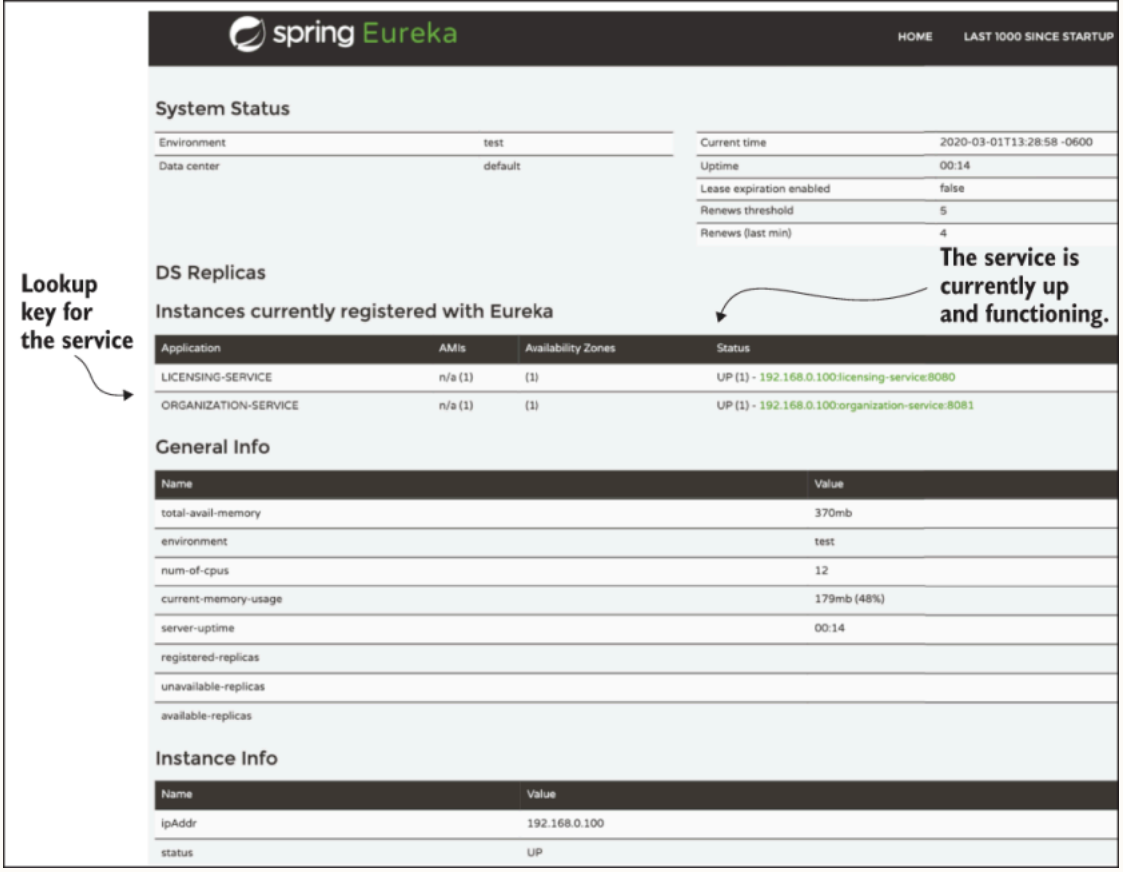


Figure 6.8 The Eureka dashboard with the registered organization and licensing instances

On Eureka and service startups: Don't be impatient

When a service registers with Eureka, Eureka waits for three successive health checks over the course of 30 seconds before the service becomes available. This warm-up period throws some developers off. They assume that Eureka hasn't registered their service if they try to call it immediately after the service is launched.

This is evident in our code examples running in the Docker environment because the Eureka service and the application services (licensing and organization) all start at the same time. Be aware that after starting an application, you might receive 404 errors about services not being found, even though the service itself has started. In that case, wait 30 seconds before trying to call your services.

In a production environment, your Eureka services will already be running. If you're deploying an existing service, the old services will still be in place to take requests.

6.5 Using service discovery to look up a service

In this section, we will explain how we can have the licensing service call the organization service without having direct knowledge of the location of any of the organization services. The licensing service will look up the physical location of the organization using Eureka.

For our purposes, we'll look at three different Spring/Netflix client libraries in which a service consumer can interact with the Spring Cloud Load Balancer. These libraries will move from the lowest level of abstraction for interacting with the Load Balancer to the highest. The libraries we'll explore include

- Spring Discovery Client
- Spring Discovery Client-enabled REST template
- Netflix Feign client

Let's walk through each of these clients and see their use in the context of the licensing service. Before we start into the specifics of the client, we wrote a few convenience classes and methods in the code. You can play with the different client types using the same service endpoint.

First, we've modified the `src/main/java/com/optimagrowth/license/controller /LicenseController.java` class to include a new route for the licensing services. This new route allows you to specify the type of client you want to use to invoke the service. This is a helper route so that as we explore each of the different methods for invoking the organization's service via the Load Balancer, you can try each mechanism through a single route. The following listing shows the code for the new route in the `LicenseController` class.

Listing 6.9 Calling the licensing service with different REST clients

```
@RequestMapping(value="/{licenseId}/{clientType}",
                method = RequestMethod.GET)          ❶
public License getLicensesWithClient(
        @PathVariable("organizationId") String organizationId,
        @PathVariable("licenseId") String licenseId,
        @PathVariable("clientType") String clientType) {
          return licenseService.getLicense(organizationId,
                licenseId, clientType);
}
```

❶ The clientType parameter determines the type of Spring REST client to use.

In listing 6.9, the `clientType` parameter passed on the route drives the type of client we're going to use in the code examples. The specific types we can pass in on this route include the following:

- *Discovery*—Uses the Discovery Client and a standard Spring `RestTemplate` class to invoke the organization service
- *Rest*—Uses an enhanced Spring `RestTemplate` to invoke the Load Balancer service
- *Feign*—Uses Netflix's Feign client library to invoke a service via the Load Balancer

NOTE Because we're using the same code for all three types of clients, you might see situations where you'll see annotations for specific clients even when they don't seem to be needed. For example, you'll see both the `@EnableDiscoveryClient` and `@EnableFeignClients` annotations in the code, even when the text is only explaining one of the client types. This is so that we can use one codebase for our examples. We'll call out these redundancies and code when these are encountered. The idea is that, as always, you choose the one that best suits your needs.

In the class `src/main/java/com/optimagrowth/license/service/LicenseService .java` , we added a simple `retrieveOrganizationInfo()` method that will resolve based on the `clientType` passed to the route. This client type is used to look up an organization service instance. The `getLicense()` method on the `LicenseService` class uses the `retrieveOrganizationInfo()` method to retrieve the organization data from a Postgres database. The following listing shows the code for the `getLicense()` service in the `LicenseService` class.

Listing 6.10 The `getLicense()` function using multiple methods for a REST call

```
public License getLicense(String licenseId, String organizationId, String clientType){
  License license = licenseRepository.findByOrganizationIdAndLicenseId
              (organizationId, licenseId);
  if (null == license) {
    throw new IllegalArgumentException(String.format(
      messages.getMessage("license.search.error.message", null, null),
      licenseId, organizationId));
  }
  Organization organization = retrieveOrganizationInfo(organizationId,
                      clientType);
  if (null != organization) {
    license.setOrganizationName(organization.getName());
    license.setContactName(organization.getContactName());
    license.setContactEmail(organization.getContactEmail());
    license.setContactPhone(organization.getContactPhone());
  }
  return license.withComment(config.getExampleProperty());
}
```

You can find each of the clients we built using the Spring Discovery Client, the Spring `RestTemplate` class, or the Feign libraries in the `src/main/java/com/optimagrowth/ license/service/client` package of the licensing service. To call the `getLicense ()` services with the different clients, you must call the following GET endpoint:

```
http://<licensing service Hostname/IP>:<licensing service Port>/v1/ organization/<organizationID>/license/<licenseID>/<client type( feign, discovery, res
```

6.5.1 Looking up service instances with Spring Discovery Client

The Spring Discovery Client offers the lowest level of access to the Load Balancer and the services registered within it. Using the Discovery Client, you can query for all the services registered with the Spring Cloud Load Balancer client and their corresponding URLs.

Next, we'll build a simple example of using the Discovery Client to retrieve one of the organization service URLs from the Load Balancer and then call the service using a standard `RestTemplate` class. To use the Discovery Client, we first need to annotate the `src/main/java/com/optimagrowth/license/LicenseServiceApplication.java` class with the `@EnableDiscoveryClient` annotation as shown in the following listing.

Listing 6.11 Setting up the bootstrap class to use the Eureka Discovery Client

```
package com.optimagrowth.license;
@SpringBootApplication
@RefreshScope
@EnableDiscoveryClient                    ❶
public class LicenseServiceApplication {

   public static void main(String[] args) {
     SpringApplication.run(LicenseServiceApplication.class, args);
   }
}
```

❶ Activates the Eureka Discovery Client

The `@EnableDiscoveryClient` is the trigger for Spring Cloud to enable the application to use the Discovery Client and the Spring Cloud Load Balancer libraries. Now, let's look at our implementation of the code that calls the organization service via the Spring Discovery Client. The following listing shows this implementation. You can find this code in the src/main/java/com/optimagrowth/license/service/client/ OrganizationDiscoveryClient.java file.

Listing 6.12 Using the Discovery Client to look up information

```
@Component
public class OrganizationDiscoveryClient {

  @Autowired
  private DiscoveryClient discoveryClient;                    ❶

  public Organization getOrganization(String organizationId) {
    RestTemplate restTemplate = new RestTemplate();
    List<ServiceInstance> instances =                         ❷
      discoveryClient.getInstances("organization-service");

    if (instances.size()==0) return null;
    String serviceUri = String.format
        ("%s/v1/organization/%s",instances.get(0)
        .getUri().toString(),
        organizationId);                                      ❸

    ResponseEntity<Organization> restExchange =               ❹
        restTemplate.exchange(
        serviceUri, HttpMethod.GET,
        null, Organization.class, organizationId);

    return restExchange.getBody();
  }
}
```

❶ Injects the Discovery Client into the class

❷ Gets a list of all the instances of the organization services

❸ Retrieves the service endpoint

❹ Uses a standard Spring RestTemplate class to call the service

The first item of interest in the code is the `DiscoveryClient` class. You use this class to interact with the Spring Cloud Load Balancer. Then, to retrieve all instances of the organization services registered with Eureka, you use the `getInstances()` method, passing in the service key that you're looking for to retrieve a list of `ServiceInstance` objects. The `ServiceInstance` class holds information about a specific instance of a service, including its hostname, port, and URI.

In listing 6.12, you take the first `ServiceInstance` class in your list to build a target URL that can then be used to call your service. Once you have a target URL, you can use a standard Spring `RestTemplate` to call your organization service and retrieve the data.

The Discovery Client and real life

You should only use the Discovery Client when your service needs to query the Load Balancer to understand what services and service instances are registered with it. There are several problems with the code in listing 6.12, including these:

- *You aren't taking advantage of the Spring Cloud client-side Load Balancer.* By calling the Discovery Client directly, you get a list of services, but it becomes your responsibility to choose which returned service instance you're going to invoke.
- *You're doing too much work.* In the code, you have to build the URL that you'll use to call your service. It's a small thing, but every piece of code that you can avoid writing is one less piece of code that you have to debug.

Observant Spring developers might have also noticed that we directly instantiated the `RestTemplate` class in the code. This is antithetical to usual Spring REST invocations because you'll usually have the Spring framework inject the `RestTemplate` class via the `@Autowired` annotation.

We instantiated the `RestTemplate` class in listing 6.12. Once we've enabled the Spring Discovery Client in the application class via `@EnableDiscoveryClient`, all REST templates managed by the Spring framework will have a Load Balancer–enabled interceptor injected into those instances. This will change how URLs are created with the `RestTemplate` class. Directly instantiating `RestTemplate` allows you to avoid this behavior.

6.5.2 Invoking services with a Load Balancer–aware Spring REST template

Next, we'll see an example of how to use a REST template that's Load Balancer–aware. This is one of the more common mechanisms for interacting with the Load Balancer via Spring. To use a Load Balancer–aware `RestTemplate` class, we need to define a `RestTemplate` bean with a Spring Cloud `@LoadBalanced` annotation.

For the licensing service, the method that we'll use to create the `RestTemplate` bean can be found in the `LicenseServiceApplication` class in src/main/java/com/ optimagrowth/license/LicenseServiceApplication.java. The following listing shows the `getRestTemplate()` method that will create the Load Balancer–backed Spring `RestTemplate` bean.

Listing 6.13 Annotating and defining a `RestTemplate` construction method

```
//Most of the import statements have been removed for conciseness
import org.springframework.cloud.client.loadbalancer.LoadBalanced;
import org.springframework.context.annotation.Bean;
import org.springframework.web.client.RestTemplate;

@SpringBootApplication
@RefreshScope
public class LicenseServiceApplication {
    public static void main(String[] args) {
      SpringApplication.run(LicenseServiceApplication.class, args);
    }

    @LoadBalanced          ❶
    @Bean
    public RestTemplate getRestTemplate(){
      return new RestTemplate();
    }
}
```

❶ Gets a list of all the instances for the organization services

Now that the bean definition for the backed `RestTemplate` class is defined, any time you want to use the `RestTemplate` bean to call a service, you only need to autowire it to the class using it.

Using the backed `RestTemplate` class pretty much behaves like a standard Spring `RestTemplate` class, except for one small difference in how the URL for the target service is defined. Rather than using the physical location of the service in the `RestTemplate` call, you need to build the target URL using the Eureka service ID of the service you want to call. The following listing allows us to see this call. The code for this listing can be found in the src/main/java/com/optimagrowth/license/service/client/OrganizationRestTemplateClient.java class file.

Listing 6.14 Using a Load Balancer-backed `RestTemplate` to call a service

```
//Package and import definitions left out for conciseness
@Component
public class OrganizationRestTemplateClient {
  @Autowired
  RestTemplate restTemplate;

  public Organization getOrganization(String organizationId){
    ResponseEntity<Organization> restExchange =
      restTemplate.exchange(
        "http://organization-service/v1/
          organization/{organizationId}",          (1)
        HttpMethod.GET, null,
        Organization.class, organizationId);

    return restExchange.getBody();
  }
}
```

(1) When using a Load Balancer-backed RestTemplate, builds the target URL with the Eureka service ID

This code should look somewhat similar to the previous example except for two key differences. First, the Spring Cloud Discovery Client is nowhere in sight, and second, the URL used in the `restTemplate.exchange ()` call should look odd to you. Here's that call:

```
restTemplate.exchange(
  "http://organization-service/v1/organization/{organizationId}",
  HttpMethod.GET, null, Organization.class, organizationId);
```

The server name in the URL matches the application ID of the organization service key that you used to register the organization service with Eureka:

```
http://{applicationid}/v1/organization/{organizationId}
```

The Load Balancer-enabled `RestTemplate` class parses the URL passed into it and uses whatever is passed in as the server name as the key to query the Load Balancer for an instance of a service. The actual service location and port are entirely abstracted from the developer. Also, by using the `RestTemplate` class, the Spring Cloud Load Balancer will round-robin load balance all requests among all the service instances.

6.5.3 Invoking services with Netflix Feign client

An alternative to the Spring Load Balancer-enabled `RestTemplate` class is Netflix's Feign client library. The Feign library takes a different approach to call a REST service. With this approach, the developer first defines a Java interface and then adds Spring Cloud annotations to map what Eureka-based service the Spring Cloud Load Balancer will invoke. The Spring Cloud framework will dynamically generate a proxy class to invoke the targeted REST service. There's no code written for calling the service other than an interface definition.

To enable the Feign client for use in our licensing service, we need to add a new annotation, `@EnableFeignClients`, to the licensing service's src/main/java/com/ optimagrowth/license/LicenseServiceApplication.java class file. The following listing shows this code.

Listing 6.15 Enabling the Spring Cloud/Netflix Feign client in the licensing service

```
@SpringBootApplication
@EnableFeignClients                    (1)
public class LicenseServiceApplication {

  public static void main(String[] args) {
    SpringApplication.run(LicenseServiceApplication.class, args);
  }
}
```

(1) This annotation is needed to use the Feign client in your code.

Now that we've enabled the Feign client for use in our licensing service, let's look at a Feign client interface definition that we can use to call an endpoint for the organization service. The following listing shows an example. You'll find the code in this listing in the src/main/java/com/optimagrowth/license/service/client/OrganizationFeignClient .java class file.

Listing 6.16 Defining a Feign interface for calling the organization service

```
//Package and import left out for conciseness
@FeignClient("organization-service")          (1)
public interface OrganizationFeignClient {
  @RequestMapping(                            (2)
    method= RequestMethod.GET,
    value="/v1/organization/{organizationId}",
    consumes="application/json")
  Organization getOrganization
    (@PathVariable("organizationId")          (3)
      String organizationId);
}
```

(1) Identifies your service to Feign

(2) Defines the path and action to your endpoint

(3) Defines the parameters passed into the endpoint

In listing 6.16, we used the `@FeignClient` annotation, passing it the application ID of the service we want the interface to represent. Then we defined a method, `getOrganization ()`, in our interface, which can be called by the client to invoke the organization service.

How we define the `getOrganization()` method looks exactly like how we would expose an endpoint in a Spring controller class. First, we define a `@RequestMapping` annotation for the `getOrganization()` method that maps the HTTP verb and endpoint to be exposed to the organization service's invocation. Second, we map the organization ID passed in on the URL to an `organizationId` parameter on the method call using `@PathVariable`. The return value from the call to the organization service is automatically mapped to the `Organization` class that's defined as the return value for the `getOrganization()` method. To use the `OrganizationFeignClient` class, all we need to do is to autowire it and use it. The Feign client code takes care of all the coding for us.

On error handling

When you use the standard Spring `RestTemplate` class, all service call HTTP status codes are returned via the `ResponseEntity` class's `getStatusCode()` method. With the Feign client, any HTTP 4xx-5xx status codes returned by the service being called are mapped to a `FeignException`. The `FeignException` contains a JSON body that can be parsed for the specific error message.

Feign provides the ability to write an error decoder class that will map the error back to a custom `Exception` class. Writing this decoder is outside the scope of this book, but you can find examples of this in the Feign GitHub repository here:

https://github.com/Netflix/feign/wiki/Custom-error-handling

Summary

- We use a service discovery pattern to abstract away the physical location of our services.
- A service discovery engine like Eureka can seamlessly add and remove service instances from an environment without impacting the service clients.

- Client-side load balancing can provide an extra level of performance and resiliency by caching the physical location of a service on the client making the service call.
- Eureka is a Netflix project that, when used with Spring Cloud, is easy to set up and configure.
- You can use these three different mechanisms in Spring Cloud and Netflix Eureka to invoke a service: Spring Cloud Discovery Client, Spring Cloud Load Balancer-backed `RestTemplate`, and Netflix's Feign client.

Previous chapter
< 5 Controlling your configuration with the Spring Cloud Configuration Server

Next chapter
7 When bad things happen: Resiliency patterns with Spring Cloud and Resilience4j >